

KONSPEKT LABORATORIUM 5

Biblioteki komponentów UI

Material UI i Tailwind / Shadcn

Kurs: Zaawansowany Interfejs Użytkownika (324796) | Czas: 90 minut | ECTS: 4 | Semestr: 1

Forma zajęć	Czas trwania	Forma zaliczenia	Efekty kształcenia
Laboratorium	90 minut (2×45 min)	Ocena z aktywności + projekt	IN2_U01, IN2_K03, IN2_K04

1. Cele zajęć

1.1 Efekty uczenia się (mapowanie)

Kod efektu	Obszar	Szczegółowy opis
IN2_U01	Umiejętności	Potrafi implementować UI używając gotowych bibliotek komponentów zgodnie z zasadami UCD.
IN2_K03	Kompetencje	Sprawnie dobiera narzędzia i biblioteki odpowiednie do charakteru projektu.
IN2_K04	Kompetencje	Potrafi komunikować techniczne decyzje projektowe wsparte kryteriami porównawczymi.

1.2 Cele operacyjne

Po zakończeniu laboratorium student:

- Potrafi zainstalować i skonfigurować Material UI v5/v6 w projekcie React + TypeScript + Vite.
- Rozumie koncepcję customowego Theme (paleta, typografia, shape, spacing) w MUI.
- Potrafi przebudować istniejące komponenty React używając komponentów MUI (Button, TextField, Checkbox, Card, Stack, Box).
- Potrafi zainstalować i skonfigurować Tailwind CSS v3 w projekcie Vite.
- Potrafi zbudować ten sam komponent używając klas utility Tailwind i porównać oba podejścia.
- Zbuduje mini-dashboard: Sidebar + Header + 3 karty statystyk w wersji MUI.
- Wypełni tabelę porównawczą MUI vs Tailwind i wyciągnie własne wnioski.

1.3 Nawiązanie do poprzednich zajęć

Kontynuacja projektu z Lab 4

Na Lab 4 studenci zaimplementowali komponenty TodoInput i TodoList używając czystego React + TypeScript + useState/useReducer. Dzisiaj przebudujemy te same komponenty — najpierw Material UI (Część A), następnie Tailwind CSS (Część B). Logika biznesowa pozostaje bez zmian — wymieniamy wyłącznie warstwę wizualną.

2. Plan zajęć (harmonogram 90 min)

Czas	Faza	Aktywność	Forma
0–5 min	Wstęp	Przypomnienie Lab 4, przegląd gotowego projektu TodoApp, przedstawienie planu sesji.	Prowadzący
5–15 min	Teoria	Wprowadzenie do bibliotek komponentów: Component Library vs CSS Framework. Krótki przegląd MUI, Tailwind, ShadCN.	Wykład + demo
15–60 min	CZĘŚĆ A	Instalacja MUI, konfiguracja Theme, przebudowa TodoInput i TodoList, mini-dashboard.	Lab A (45 min)
60–75 min	CZĘŚĆ B	Instalacja Tailwind, konfiguracja, przebudowa tych samych dwóch komponentów klasami utility.	Lab B (15 min)
75–85 min	Analiza	Wypełnienie tabeli porównawczej MUI vs Tailwind. Dyskusja grupowa.	Studenci
85–90 min	Podsumowanie	Prezentacja wyników, ocena aktywności, informacje o Lab 6 (Responsive Design).	Prowadzący

3. Instrukcja instalacji i konfiguracji bibliotek

3.1 Material UI v6 (MUI)

Wykonaj poniższe kroki w terminalu, w katalogu projektu z Lab 4:

Krok 1 — Instalacja pakietów MUI

```
# Podstawowe pakiety MUI v6
npm install @mui/material @emotion/react @emotion/styled

# Ikony MUI (używane w mini-dashboardzie)
npm install @mui/icons-material

# Czcionki Roboto (zalecane przez MUI)
npm install @fontsource/roboto
```

Krok 2 — Import czcionek Roboto w main.tsx

```
// src/main.tsx – dodaj na samej górze pliku
import '@fontsource/roboto/300.css';
import '@fontsource/roboto/400.css';
import '@fontsource/roboto/500.css';
import '@fontsource/roboto/700.css';
```

Krok 3 — Opcjonalna optymalizacja vite.config.ts

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    include: ['@mui/material', '@emotion/react', '@emotion/styled'],
  },
});
```

```
    },  
  });  
};
```

3.2 Tailwind CSS v3

Krok 1 — Instalacja Tailwind i zależności

```
npm install -D tailwindcss postcss autoprefixer  
npx tailwindcss init -p
```

Krok 2 — Konfiguracja tailwind.config.js

```
// tailwind.config.js  
/** @type {import('tailwindcss').Config} */  
export default {  
  content: [  
    './index.html',  
    './src/**/*.{js,ts,jsx,tsx}',  
  ],  
  theme: {  
    extend: {  
      colors: {  
        brand: {  
          50: '#E3F2FD',  
          100: '#BBDEFB',  
          500: '#1565C0',  
          700: '#0D47A1',  
        },  
        success: '#2E7D32',  
        warning: '#E65100',  
        danger: '#B71C1C',  
      },  
      fontFamily: {  
        sans: ['Inter', 'system-ui', 'sans-serif'],  
      },  
      borderRadius: {  
        'x1': '1rem',  
        '2x1': '1.5rem',  
      },  
    },  
  },  
  plugins: [],  
};
```

Krok 3 — Import Tailwind w głównym pliku CSS

```
/* src/index.css - zastąp całą zawartość */  
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

4. Część A (45 min) — Przebudowa UI z Lab 4 przy użyciu Material UI

4.1 Customowy Theme MUI — gotowy kod

Utwórz plik `src/theme/muiTheme.ts` i wklej poniższy kod. Theme definiuje kolory marki, typografię oraz zaokrąglenia elementów:

```
// src/theme/muiTheme.ts
import { createTheme } from '@mui/material/styles';

const muiTheme = createTheme({
  palette: {
    primary: {
      main: '#1565C0', // brand blue
      light: '#1E88E5',
      dark: '#0D47A1',
      contrastText: '#FFFFFF',
    },
    secondary: {
      main: '#E65100', // accent orange
      light: '#FF8A65',
      dark: '#BF360C',
    },
    success: { main: '#2E7D32' },
    error: { main: '#B71C1C' },
    background: {
      default: '#F5F7FA',
      paper: '#FFFFFF',
    },
  },
  typography: {
    fontFamily: '"Roboto", "Helvetica", "Arial", sans-serif',
    h4: { fontWeight: 700, letterSpacing: '-0.02em' },
    h5: { fontWeight: 600 },
    h6: { fontWeight: 600 },
    button: { textTransform: 'none', fontWeight: 600 },
  },
  shape: {
    borderRadius: 10,
  },
  spacing: 8,
  components: {
    MuiButton: {
      defaultProps: { disableElevation: true },
      styleOverrides: {
        root: { borderRadius: 8, paddingLeft: 20, paddingRight: 20 },
      },
    },
    MuiTextField: {
      defaultProps: { variant: 'outlined', size: 'small' },
    },
    MuiCard: {
      styleOverrides: {
        root: { boxShadow: '0 2px 12px rgba(0,0,0,0.08)', borderRadius: 12 },
      },
    },
  },
});

export default muiTheme;
```

4.2 Opakowanie aplikacji w ThemeProvider

```
// src/main.tsx
import { ThemeProvider, CssBaseline } from '@mui/material';
import muiTheme from './theme/muiTheme';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <ThemeProvider theme={muiTheme}>
      <CssBaseline />
      <App />
    </ThemeProvider>
  </React.StrictMode>
);
```

4.3 Przebudowa komponentu TodoInput — wersja MUI

```
// src/components/TodoInput.tsx (wersja MUI)
import { useState } from 'react';
import { Box, TextField, Button, Typography } from '@mui/material';
import AddIcon from '@mui/icons-material/Add';

interface TodoInputProps {
  onAdd: (text: string) => void;
}

export default function TodoInput({ onAdd }: TodoInputProps) {
  const [text, setText] = useState('');

  const handleSubmit = () => {
    if (!text.trim()) return;
    onAdd(text.trim());
    setText('');
  };

  return (
    <Box sx={{ mb: 3 }}>
      <Typography variant='h6' sx={{ mb: 1 }}>
        Dodaj nowe zadanie
      </Typography>
      <Box sx={{ display: 'flex', gap: 1 }}>
        <TextField
          fullWidth
          placeholder='Wpisz treść zadania...'
          value={text}
          onChange={e => setText(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && handleSubmit()}
        />
        <Button
          variant='contained'
          startIcon={<AddIcon />}
          onClick={handleSubmit}
          disabled={!text.trim()}
        >
          Dodaj
        </Button>
      </Box>
    </Box>
  );
}
```

4.4 Przebudowa komponentu `ToDoList` — wersja MUI

```
// src/components/ToDoList.tsx (wersja MUI)
import {
  List, ListItem, ListItemText, ListItemIcon,
  Checkbox, IconButton, Typography, Paper, Chip
} from '@mui/material';
import DeleteOutlineIcon from '@mui/icons-material/DeleteOutline';
import { Todo } from '../types/todo';

interface ToDoListProps {
  todos: Todo[];
  onToggle: (id: string) => void;
  onDelete: (id: string) => void;
}

export default function ToDoList({ todos, onToggle, onDelete }: ToDoListProps) {
  if (todos.length === 0) {
    return (
      <Typography color='text.secondary' textAlign='center' sx={{ mt: 4 }}>
        Brak zadań. Dodaj pierwsze!
      </Typography>
    );
  }
  return (
    <Paper variant='outlined' sx={{ overflow: 'hidden' }}>
      <List disablePadding>
        {todos.map((todo, idx) => (
          <ListItem
            key={todo.id}
            divider={idx < todos.length - 1}
            sx={{ bgcolor: todo.completed ? 'action.hover' : 'background.paper' }}
            secondaryAction={
              <IconButton
                edge='end'
                color='error'
                onClick={() => onDelete(todo.id)}
                aria-label='Usuń zadanie'
              >
                <DeleteOutlineIcon />
              </IconButton>
            }
          >
            <ListItemIcon>
              <Checkbox
                checked={todo.completed}
                onChange={() => onToggle(todo.id)}
                inputProps={{ 'aria-label': todo.text }}
              />
            </ListItemIcon>
            <ListItemText
              primary={todo.text}
              sx={{
                textDecoration: todo.completed ? 'line-through' : 'none',
                color: todo.completed ? 'text.disabled' : 'text.primary',
              }}
            />
            {todo.completed && (
              <Chip label='Ukończone' size='small' color='success' sx={{ mr: 1 }} />
            )}
          </ListItem>
        ))}
      </List>
    </Paper>
  );
}
```

```
}
```

TODO 1 — zadanie w kodzie

W komponencie `TodoList` dopisz prop `'filter'` (typ: `'all' | 'active' | 'completed'`) i filtruj listę przed renderowaniem. Użyj `useMemo()` dla wydajności. Czas: ok. 10 min.

5. Mini-projekt sesji — Dashboard MUI (Sidebar + Header + Karty statystyk)

5.1 Opis funkcjonalny

Studenci budują autonomiczny ekran Dashboard wewnątrz aplikacji `TodoApp`. Dashboard zawiera:

- Sidebar nawigacyjny z logo, listą linków (Dashboard, Zadania, Ustawienia) i avatarą użytkownika.
- Header z tytułem strony, przyciskiem motywu (dark/light placeholder) i ikoną powiadomień.
- 3 karty statystyk: Wszystkie zadania, Ukończone, Oczekujące.
- Dane do kart są obliczane dynamicznie na podstawie stanu kontekstu (`TodoContext` z Lab 4).

5.2 Struktura plików

```
src/
  components/
    dashboard/
      DashboardLayout.tsx // główny layout: Sidebar + main area
      Sidebar.tsx         // nawigacja boczna — MUI Drawer
      AppHeader.tsx       // górny pasek — MUI AppBar
      StatsCard.tsx       // karta z pojedynczą statystyką
      StatsGrid.tsx       // siatka 3 kart (używa StatsCard)
```

5.3 Komponent StatsCard — gotowy kod

```
// src/components/dashboard/StatsCard.tsx
import { Card, CardContent, Typography, Box, Avatar } from '@mui/material';
import { SvgIconComponent } from '@mui/icons-material';

interface StatsCardProps {
  title: string;
  value: number;
  icon: SvgIconComponent;
  color: string;
  bgColor: string;
}

export default function StatsCard({ title, value, icon: Icon, color, bgColor }: StatsCardProps) {
  return (
    <Card>
      <CardContent>
```

```
    <Box sx={{ display: 'flex', justifyContent: 'space-between', alignItems:
'flex-start' }}>
      <Box>
        <Typography variant='body2' color='text.secondary' gutterBottom>
          {title}
        </Typography>
        <Typography variant='h4' fontWeight={700}>
          {value}
        </Typography>
      </Box>
      <Avatar sx={{ bgcolor: bgColor, color, width: 48, height: 48 }}>
        <Icon />
      </Avatar>
    </Box>
  </CardContent>
</Card>
);
}
```

5.4 Komponent StatsGrid — szkielet do uzupełnienia

```
// src/components/dashboard/StatsGrid.tsx
import { Grid } from '@mui/material';
import FormatListBulletedIcon from '@mui/icons-material/FormatListBulleted';
import CheckCircleIcon from '@mui/icons-material/CheckCircle';
import RadioButtonUncheckedIcon from '@mui/icons-material/RadioButtonUnchecked';
import StatsCard from '../StatsCard';
import { useTodoContext } from '../../../context/ToDoContext';

export default function StatsGrid() {
  const { state } = useTodoContext();

  // TODO 2: Oblicz wartości na podstawie state.todos
  const total = 0; // UZUPEŁNIJ: liczba wszystkich zadań
  const completed = 0; // UZUPEŁNIJ: liczba ukończonych
  const pending = 0; // UZUPEŁNIJ: liczba oczekujących

  return (
    <Grid container spacing={3}>
      <Grid item xs={12} sm={4}>
        <StatsCard
          title='Wszystkie zadania'
          value={total}
          icon={FormatListBulletedIcon}
          color='#1565C0'
          bgColor='#E3F2FD'
        />
      </Grid>
      {/* TODO 3: Dodaj karty dla Ukończone i Oczekujące */}
    </Grid>
  );
}
```

5.5 Komponent Sidebar — szkielet

```
// src/components/dashboard/Sidebar.tsx
import {
  Drawer, List, ListItemButton, ListItemIcon,
```



```
    ListItemText, Toolbar, Typography, Divider, Avatar, Box
} from '@mui/material';
import DashboardIcon from '@mui/icons-material/Dashboard';
import TaskIcon      from '@mui/icons-material/Task';
import SettingsIcon  from '@mui/icons-material/Settings';

const DRAWER_WIDTH = 240;

const navItems = [
  { label: 'Dashboard', icon: DashboardIcon, path: '/' },
  { label: 'Zadania', icon: TaskIcon, path: '/todos' },
  { label: 'Ustawienia', icon: SettingsIcon, path: '/settings' },
];

export default function Sidebar() {
  return (
    <Drawer
      variant='permanent'
      sx={{
        width: DRAWER_WIDTH,
        '& .MuiDrawer-paper': {
          width: DRAWER_WIDTH,
          boxSizing: 'border-box',
          bgcolor: 'primary.main',
          color: 'white',
        },
      }}
    >
      <Toolbar>
        <Typography variant='h6' fontWeight={700}>TodoApp</Typography>
      </Toolbar>
      <Divider sx={{ borderColor: 'rgba(255,255,255,0.2)' }} />
      <List>
        {navItems.map(item => (
          // TODO 4: Renderuj ListItemButton dla każdego navItem
          // Użyj item.icon jako komponent w ListItemIcon
          null
        ))}
      </List>
      <Box sx={{ flexGrow: 1 }} />
      <Box sx={{ p: 2, display: 'flex', alignItems: 'center', gap: 1 }}>
        <Avatar sx={{ width: 36, height: 36, bgcolor: 'primary.dark' }}>U</Avatar>
        <Typography variant='body2'>Użytkownik</Typography>
      </Box>
    </Drawer>
  );
}
```

5.6 DashboardLayout — gotowy layout

```
// src/components/dashboard/DashboardLayout.tsx
import { Box, Toolbar } from '@mui/material';
import Sidebar          from './Sidebar';
import AppHeader        from './AppHeader';
import StatsGrid        from './StatsGrid';

export default function DashboardLayout() {
  return (
    <Box sx={{ display: 'flex', minHeight: '100vh' }}>
      <Sidebar />
      <Box component='main' sx={{ flexGrow: 1, p: 3, bgcolor: 'background.default' }}>
        <AppHeader />
      </Box>
    </Box>
  );
}
```

```
        <Toolbar />
        <StatsGrid />
      </Box>
    </Box>
  );
}
```

TODO 5 — rozszerzenie (dla chętnych)

Dodaj do DashboardLayout sekcję z ostatnimi zadaniami: użyj komponentu Timeline z @mui/lab (npm install @mui/lab) pokazując 5 ostatnio dodanych zadań posortowanych po dacie. Czas: ok. 15 min.

6. Część B (15 min) — Przebudowa 2 komponentów przy użyciu Tailwind CSS

6.1 Cel ćwiczenia

Przebuduj wyłącznie komponenty TodoInput i TodoList używając klas Tailwind (bez komponentów MUI). Logika biznesowa i props pozostają bez zmian. Celem jest doświadczenie obu podejść na tym samym problemie.

6.2 TodoInput — wersja Tailwind

```
// src/components/ToDoInput.tailwind.tsx
import { useState } from 'react';

interface TodoInputProps {
  onAdd: (text: string) => void;
}

export default function TodoInputTailwind({ onAdd }: TodoInputProps) {
  const [text, setText] = useState('');

  const handleSubmit = () => {
    if (!text.trim()) return;
    onAdd(text.trim());
    setText('');
  };

  return (
    <div className='mb-6'>
      <h2 className='text-lg font-semibold text-gray-800 mb-2'>
        Dodaj nowe zadanie
      </h2>
      <div className='flex gap-2'>
        <input
          type='text'
          placeholder='Wpisz treść zadania...'
          value={text}
          onChange={e => setText(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && handleSubmit()}
          className='flex-1 px-4 py-2 text-sm border border-gray-300 rounded-lg
            focus:outline-none focus:ring-2 focus:ring-brand-500 focus:border-
transparent
            placeholder:text-gray-400'
```

```
    />
    <button
      onClick={handleSubmit}
      disabled={!text.trim()}
      className='px-5 py-2 text-sm font-semibold text-white bg-brand-500 rounded-
lg
      hover:bg-brand-700 transition-colors disabled:opacity-40 disabled:cursor-
not-allowed'
    >
      Dodaj
    </button>
  </div>
</div>
);
}
```

6.3 TodoList — szkielet Tailwind do uzupełnienia

```
// src/components/ToDoList.tailwind.tsx
import { Todo } from '../types/todo';

interface ToDoListProps {
  todos: Todo[];
  onToggle: (id: string) => void;
  onDelete: (id: string) => void;
}

export default function ToDoListTailwind({ todos, onToggle, onDelete }: ToDoListProps)
{
  if (todos.length === 0) {
    return (
      <p className='text-center text-gray-400 mt-8'>Brak zadań. Dodaj pierwsze!</p>
    );
  }
  return (
    <ul className='divide-y divide-gray-200 border border-gray-200 rounded-xl
overflow-hidden'>
      {todos.map(todo => (
        <li
          key={todo.id}
          className={`flex items-center gap-3 px-4 py-3 ${todo.completed ? 'bg-gray-
50' : 'bg-white'} `}
        >
          {/* TODO 6: Dodaj <input type='checkbox'> z klasami Tailwind */}
          {/* TODO 7: Dodaj <span> z tekstem, przekreślonym gdy completed */}
          {/* TODO 8: Dodaj przycisk usuwania po prawej stronie */}
        </li>
      ))}
    </ul>
  );
}
```

7. Tabela porównawcza do uzupełnienia — MUI vs Tailwind

Instrukcja: Po wykonaniu obu części laboratorium uzupełnij tabelę na podstawie własnych doświadczeń. Użyj skali 1–5 lub krótkich opisów słownych.

Kryterium	Material UI (MUI)	Tailwind CSS	Zwycięzca?
DX – szybkość startu			...
DX – czytelność kodu			...
Bundle size (prod)			...
Customizacja stylów			...
Wsparcie dla TypeScript			...
Dostępność (a11y) out-of-box			...
Responsive design (utility)			...
Theming / design tokens			...
Krzywa uczenia się			...
Integracja z Figma / DS			...

7.1 Kluczowe spostrzeżenia — wniosek studenta

Kiedy wybrać MUI?

.....

Kiedy wybrać Tailwind?

.....

Kiedy łączyć obie biblioteki (MUI + Tailwind)?

.....

8. Rozszerzenia dla studentów zaawansowanych

8A. ShadCN/UI — trzecia opcja

ShadCN/UI to kolekcja komponentów zbudowanych na Radix UI + Tailwind CSS. Komponenty są kopiowane bezpośrednio do projektu — brak zależności npm do zainstalowania.

- Instalacja: `npx shadcn@latest init`
- Dodanie przycisku: `npx shadcn@latest add button`
- Komponenty trafiają do `src/components/ui/` i można je dowolnie modyfikować.
- Zadanie: Przebuduj przycisk Dodaj w TodoInput używając Button z ShadCN i porównaj z MUI oraz Tailwind.

8B. Dark mode w MUI

.....

```
// Dynamiczna zmiana motywu – dodaj do App.tsx
import { useState } from 'react';
import { ThemeProvider, createTheme } from '@mui/material/styles';
import { CssBaseline, IconButton } from '@mui/material';
import DarkModeIcon from '@mui/icons-material/DarkMode';

export default function App() {
  const [mode, setMode] = useState<'light' | 'dark'>('light');

  const theme = createTheme({ palette: { mode } });

  return (
    <ThemeProvider theme={theme}>
      <CssBaseline />
      <IconButton onClick={() => setMode(m => m === 'light' ? 'dark' : 'light')}>
        <DarkModeIcon />
      </IconButton>
      { /* reszta aplikacji */ }
    </ThemeProvider>
  );
}
```

9. Kryteria oceny laboratorium

Zajęcia są zaliczone, gdy spełnione są wymagania minimalne. Poniżej przedstawiono pełne spektrum osiągnięć — od zaliczenia z wyróżnieniem do niezaliczenia.

Poziom	Wymagania	Status
Z wyróżnieniem	Część A: komponenty MUI + customowy Theme + mini-dashboard z dynamicznymi danymi + TODO 1–5 ukończone. Część B: TodoInput i TodoList Tailwind gotowe. Tabela porównawcza uzupełniona z własnymi wnioskami.	✓
Dobrze	Część A z Theme i mini-dashboarde (bez dynamicznych danych). Część B: przynajmniej TodoInput Tailwind. Tabela porównawcza częściowo uzupełniona.	✓
Wymagania minimalne	Przynajmniej jeden komponent MUI (TodoInput) z działającym ThemeProvider. Część B i dashboard mogą być niepełne.	✓
NIEZALICZONE	Brak instalacji MUI lub brak jakichkolwiek działających komponentów bibliotecznych. Aplikacja nie uruchamia się lub nie zawiera żadnej zmiany względem Lab 4.	X

10. Materiały i literatura

Dokumentacja oficjalna (obowiązkowa)

- Material UI v6: <https://mui.com/material-ui/getting-started/>
- MUI Theming: <https://mui.com/material-ui/customization/theming/>
- Tailwind CSS v3: <https://tailwindcss.com/docs>
- ShadCN/UI: <https://ui.shadcn.com/docs>

Materiały uzupełniające

- MUI System – sx prop: <https://mui.com/system/getting-started/>
- Tailwind CSS Cheat Sheet: <https://nerdcave.com/tailwind-cheat-sheet>
- Porównanie bibliotek CSS 2024: <https://stateofcss.com>

Nawiązanie do następnych zajęć

Lab 6 — Responsive Design i Adaptacja Interfejsów

Na kolejnych zajęciach rozszerzymy komponenty MUI i Tailwind o pełną responsywność: Grid system MUI (xs/sm/md/lg), Tailwind breakpoints (sm:/md:/lg:), useMediaQuery hook, adaptacja Sidebar do trybu mobilnego (hamburger menu). Projekt TodoApp zostanie dostosowany do ekranów mobilnych.